

Milestone 2

Objective: Movie Revenue Prediction

The goal of this project is to develop an **Interactive dashboard** based on machine learning model that accurately predicts a movie's box office revenue based on various features such as budget, genre, director, cast, critic/audience ratings, and historical box office performance.

Our aim is to identify major factors that influence a film's success at the Box Office, by leveraging data from sources like IMDb, TMDb, and Rotten Tomatoes.

Tech Stack:

- **Python:** Main programming language for data processing & modeling
- **Pandas, NumPy, SciPy, sklearn:** Data manipulation and preprocessing
- **SQLite:** Store cleaned and structured data
- **Matplotlib, Seaborn, Plotly:** Data visualization and EDA
- **Scikit-Learn, Pytorch:** Machine Learning
- **Flask:** API for serving predictions
- **Streamlit:** Interactive web-based dashboard for model insights

Audience: filmmakers, producers, and investors.

Project Timeline:

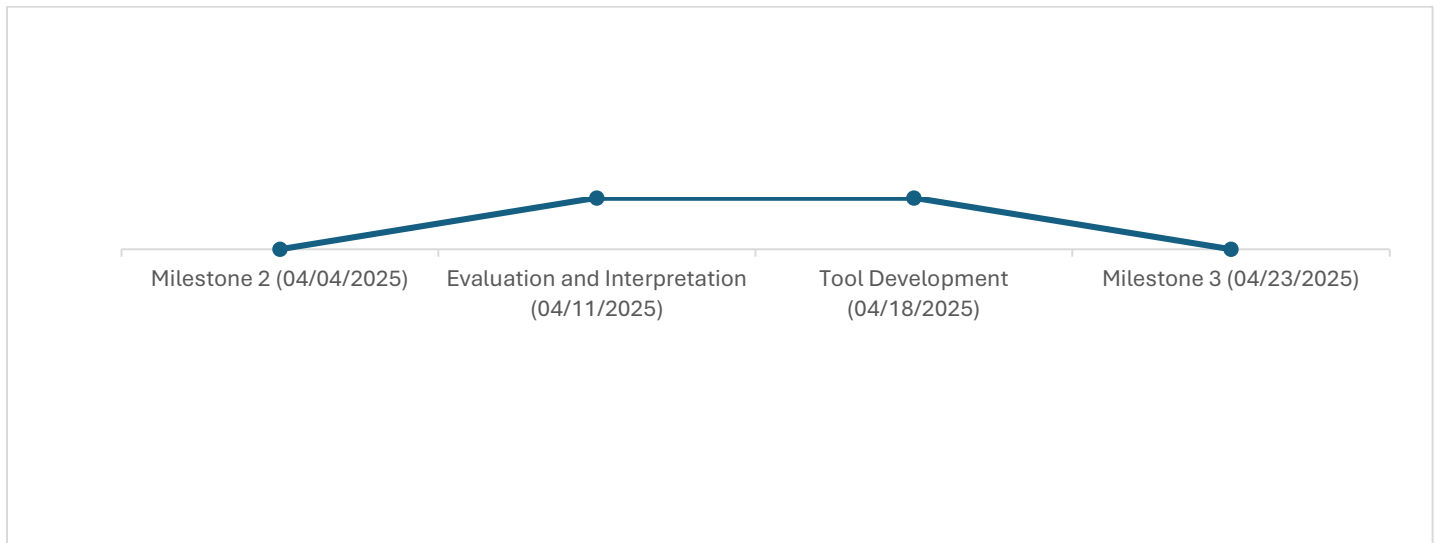


Figure 1 Project Timeline

Dataset Description:

Explored By	Dataset	Source	Variables	Dimension
Mohammad Wael	TMDb	Kaggle	id, title, vote_average, vote_count, status, release_date, revenue, runtime, adult, backdrop_path, budget, homepage, imdb_id, original_language, original_title, overview, popularity, poster_path, tagline, genres, production_companies, production_countries, spoken_languages, keywords	1181912 Rows, 24 Columns
	Rotten Tomatoes Critic	Kaggle	rotten_tomatoes_link, critic_name, top_critic, publisher_name, review_type, review_score, review_date, review_content	1130017 Rows, 8 Columns

	Rotten Tomatoes Movie		rotten_tomatoes_link, movie_title, movie_info, critics_consensus, content_rating, genres, directors, authors, actors, original_release_date, streaming_release_date, runtime, production_company, tomatometer_status, tomatometer_rating, tomatometer_count, audience_status, audience_rating, audience_count, tomatometer_top_critics_count, tomatometer_fresh_critics_count, tomatometer_rotten_critics_count	17712 Rows, 22 Columns
	Title.Basics		tconst, titleType, primaryTitle, originalTitle, isAdult, startYear, endYear, runtimeMinutes, genres,	11464895 Rows, 9 Columns
Vishal K. Setti	Name.Basics	IMDb	nconst, primaryName, birthYear, deathYear, primaryProfession, knownForTitles	14195120 Rows, 6 Columns
	Title.Rating		tconst, averageRating, numVotes	1535911 Rows, 3 Columns
	Title.Principals		tconst, ordering, nconst, category, job, characters	90984102 Rows, 6 Columns
	Title.Crew		tconst, directors, writers	11464895 rows, 3 Columns
	ISO2 codes to ISO3	Kaggle	country, alpha2, alpha3, numeric	249 Rows, 4 Columns

Table 1 Dataset Description

Feature Engineering & Encoding Methodology

1. Novel Feature Creation

We engineered multiple sophisticated predictive features beyond basic transformations, strategically categorized to capture different dimensions of movie performance:

- a. **Time-based features** are variables derived from date or time information in a dataset, such as year, month, day, hour, day of the week, or whether it's a weekend or holiday. These features are especially valuable in time-sensitive applications like forecasting, sales analysis, or user behavior modeling.

They help capture patterns, trends, and seasonality in the data. For example:

- **Month or season** can reveal seasonal effects on sales.
- **Day of the week** might indicate different customer behavior on weekdays vs. weekends.
- **Hour of the day** can be crucial for modeling traffic or usage spikes.

By extracting and incorporating time-based features, models can better understand temporal dynamics and make more accurate predictions.

```
revenue_data = revenue_data.rename(columns={
    "year" : "release_year",
    "month" : "release_month"
})

revenue_data["is_festive_month"] =
revenue_data["release_month"].apply(is_festive)
```

Feature	Formula	Justification
release_month	Extracted from date	Captures seasonal box office patterns (e.g., festive season)
release_year		

Table 2 Time-Based Features

b. Numerical ratio features are quantitative variables that represent meaningful ratios between values, typically derived by dividing one numerical feature by another. Examples include profit margin (profit/revenue), click-through rate (clicks/impressions), or average rating (total score/number of ratings).

These features are important because:

- They **normalize** data, making it comparable across different scales or sizes (e.g., comparing revenue per user rather than total revenue).
- They often reveal **proportional relationships** and trends that raw values might not show.
- They can help the model focus on **relative performance**, which is often more informative than absolute figures.

Using well-thought-out ratio features can enhance model accuracy and interpretability by highlighting the underlying patterns in the data.

```
# budget based features
omdbc["budget_vote_ratio"] = omdbc["budget"] / (omdbc["votes"] + 1)
omdbc["budget_runtime_ratio"] = omdbc["budget"] / (omdbc["runtime"] + 1)
omdbc["budget_score_ratio"] = omdbc["log_budget"] / (omdbc["score"] + 1)
omdbc["budget_year_ratio"] = omdbc["log_budget"] / (omdbc["year"] -
omdbc["year"].min() + 1)
omdbc["budget_per_minute"] = omdbc["budget"] / (omdbc["runtime"] + 1)

# votes based features
omdbc["vote_score_ratio"] = omdbc["votes"] / (omdbc["score"] + 1)
omdbc["votes_per_year"] = omdbc["votes"] / (omdbc["year"] - omdbc["year"].min() +
1)
omdbc["vote_year_ratio"] = omdbc["votes"] / (omdbc["year"] - omdbc["year"].min() +
1)

#score based features
omdbc["score_runtime_ratio"] = omdbc["score"] / (omdbc["runtime"] + 1)
```

Feature	Formula	Business Logic
vote_score_ratio	votes / (score + 1)	Identifies "controversial" or divisive films with high engagement despite mixed reviews
budget_per_minute	budget / runtime	Measures production value density and resource allocation efficiency
score_runtime_ratio	score / runtime	Quantifies entertainment value per minute, indicating pacing effectiveness
budget_year_ratio	budget / (year - min_year + 1)	Normalizes budget across different film eras
vote_year_ratio	votes / (release year age)	Accounts for voting accumulation over time
votes_per_year	votes / years since release	Measures sustained audience engagement velocity

Table 3 Numerical Ratio Features

c. **Boolean threshold features** are binary (True/False or 1/0) indicators created by applying a condition or threshold to a numerical or categorical feature. For example, creating a feature like `is_high_rating = rating > 4` turns a continuous value into a simple yes/no flag.

These features are important because:

- They **simplify complex data** into easily interpretable conditions.
- They help models **capture specific rules or patterns**, such as whether a movie is considered highly rated or a product is expensive.
- They can improve model performance by focusing on **key decision points** instead of full value ranges.

Boolean threshold features are especially useful in tree-based models or rule-based systems, where clear-cut conditions help split data meaningfully

```
# considering the year, budget, votes and score as features and values greater than
75 percentile as high values

year_q3 = omdbc['year'].quantile(0.75)
omdbc['is_recent'] = (omdbc['year'] >= year_q3).astype(int)

budget_q3 = omdbc['budget'].quantile(0.75)
omdbc['is_high_budget'] = (omdbc['budget'] >= budget_q3).astype(int)

votes_q3 = omdbc['votes'].quantile(0.75)
omdbc['is_high_votes'] = (omdbc['votes'] >= votes_q3).astype(int)

score_q3 = omdbc['score'].quantile(0.75)
omdbc['is_high_score'] = (omdbc['score'] >= score_q3).astype(int)
```

Feature	Calculation	Strategic Value
is_high_budget	budget ≥ 75th percentile	Flags potential blockbusters and high-risk productions
is_high_votes	votes ≥ 75th percentile	Indicates exceptional audience reach and engagement
is_recent	release within last 5 years	Captures contemporary market dynamics
is_high_score	score ≥ 7.5	Identifies critical acclaim threshold

Table 4 Boolean Features

Justification: These engineered features capture complex non-linear relationships and domain-specific insights that raw features alone cannot express:

- `budget_year_ratio` adjusts for changing economic contexts across different eras
- `vote_score_ratio` helps detect potential "cult classics" with passionate but divided audiences
- Boolean features create decision-ready segmentation for strategic categorization

2. Categorical Variable Encoding

Variable Type	Method	Rationale
Genre, Director, etc.	Label Encoding	15 main genres (low cardinality), non-exclusive categories
	One Hot encoding	Used before running SelectKBest to find the important features

Table 5 Categorical Variable Encoding

Label Encoder:

To prepare the data for regression models, it's essential to convert non-numerical (categorical) features into numerical values. For this, we use the LabelEncoder from the sklearn.preprocessing module in scikit-learn.

The LabelEncoder transforms categorical labels into numeric form by assigning each unique category a different integer. For example, a feature with categories like ["red", "green", "blue"] would be encoded as [0, 1, 2]. This is useful for algorithms that require numerical input, although it's best suited for ordinal categories or target labels rather than features with no intrinsic order.

```
def encode_categorical_features(df):
    """Encode categorical features using label encoding"""
    categorical_features = [
        "released", "writer", "rating", "name",
        "genre", "director", "star", "country", "company"
    ]

    # Only encode columns that exist in DataFrame
    existing_cats = [col for col in categorical_features if col in df.columns]

    for feature in existing_cats:
        le = LabelEncoder()
        df[feature] = le.fit_transform(df[feature].astype(str))

    return df
```

One Hot Encoder:

This code processes categorical features from the omdbc dataset in preparation for machine learning. It first selects specific categorical columns (like "released", "writer", "genre", etc.) and then applies One-Hot Encoding using scikit-learn's OneHotEncoder.

One-Hot Encoding converts each categorical feature into a set of binary (0 or 1) columns — one for each unique category — indicating the presence of that category in a given row. For example, a genre column with values ["Action", "Comedy"] becomes two columns: genre_Action and genre_Comedy.

The sparse=False parameter ensures the encoded output is a dense NumPy array, and handle_unknown="ignore" prevents errors if unseen categories appear in the test data.

Finally, the code generates human-readable names for the new one-hot encoded columns using get_feature_names_out, and also ensures the column names in the numerical features are strings for consistency.

```
    # Process categorical features
categorical_features = omdbc[
    [
        "released",
        "writer",
        "rating",
        "name",
        "genre",
        "director",
        "star",
        "company",
    ]
]
```

```
# One-hot encode categorical features
encoder = OneHotEncoder(sparse=False, handle_unknown="ignore")
encoded_categorical = encoder.fit_transform(categorical_features)

# Prepare column names
numerical_features.columns = numerical_features.columns.astype(str)
categorical_feature_names = [
    str(col) for col in
encoder.get_feature_names_out(categorical_features.columns)
]
```

These engineered features and encoding strategies create a rich, multidimensional representation of movie attributes, capturing both explicit and latent factors that influence box office performance. The combination of ratio-based features, binary indicators, and appropriate encoding methods provides a robust foundation for predictive modeling across multiple algorithms.

Exploratory Data Analysis (EDA) for New Features

Performed EDA on newly created features helps validate their usefulness and identify relationships with the target variable (movie revenue).

1. Summary Statistics

```
new_features = [
    'budget_year_ratio', 'vote_score_ratio',
    'score_runtime_ratio', 'is_high_budget'
]

print(omdbc[new_features].describe())
```

Output:

Statistic	budget_year_ratio	vote_score_ratio	score_runtime_ratio	is_high_budget
count	5,421.00	5,421.00	5,421.00	5,421.00
mean	0	4.19E-17	-2.20E-16	2.10E-17
std	1	1	1	1
min	-0.54	-0.7	-3.95	-0.58
25%	-0.4	-0.57	-0.64	-0.58
50%	-0.28	-0.35	-0.05	-0.58
75%	-0.03	0.13	0.61	1.71
max	9.81	10.98	5.2	1.71

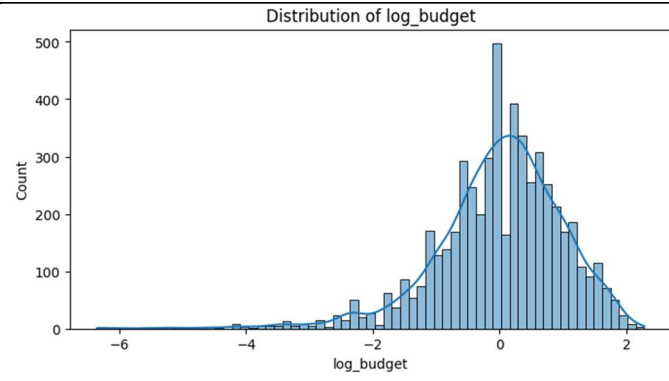
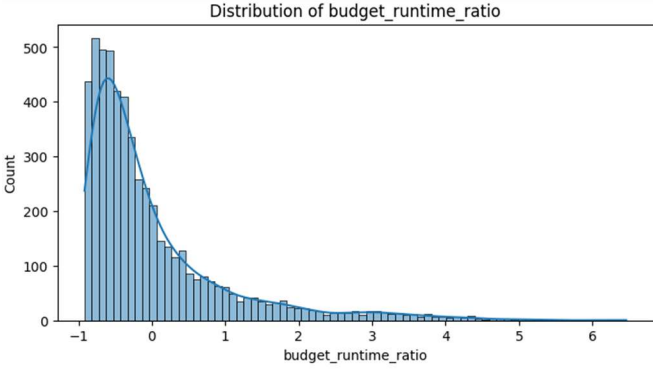
Table 6 Summary Stats for New Features

Observations:

- **Standardized Features:** All four engineered features have been standardized (z-score normalized) with means near zero and standard deviations of 1.0, enabling fair comparison of their effects in modeling.
- **Binary High Budget Feature:** The **is_high_budget** feature shows a clear binary distribution with approximately 75% of movies classified as not high budget (-0.58) and 25% classified as high budget (1.71), matching the 75th percentile threshold design.
- **Right-Skewed Distributions:** Both **budget_year_ratio** and **vote_score_ratio** show significant positive skew with maximum values (9.81 and 10.98) far exceeding the 75th percentiles (-0.03 and 0.13), indicating the presence of extreme outliers - likely blockbuster films with exceptional budgets or controversial films with disproportionate engagement.
- **Balanced Score-Runtime Relationship:** The **score_runtime_ratio** shows the most balanced distribution among the continuous features, with relatively symmetric quartiles around the median, suggesting that entertainment efficiency varies more naturally across films.
- **Outlier Management Opportunity:** The large gaps between 75th percentiles and maximum values for **budget_year_ratio** and **vote_score_ratio** suggest that outlier treatment could improve model stability, particularly for algorithms sensitive to extreme values.

2. Distribution Analysis for Key features

```
for feature in numerical_features:
    plt.figure(figsize=(8, 4))
    sns.histplot(omdbc[feature], kde=True)
    plt.title(f"Distribution of {feature}")
    plt.show()
```

Plot	Observations
 The plot shows a histogram of the log_budget feature. The x-axis is labeled 'log_budget' and ranges from -6 to 2. The y-axis is labeled 'Count' and ranges from 0 to 500. The distribution is approximately normal, centered around 0, with a slight right skew. A KDE curve is overlaid on the histogram.	Normal Distribution Achievement: The logarithmic transformation has successfully normalized the highly skewed raw budget data, creating an approximately normal distribution that is more suitable for statistical modeling techniques that assume normality. Centered Distribution: The distribution is centered slightly below zero, indicating that the "average" film budget (after log transformation) falls just below the reference point, likely representing the industry median budget. Right Skew Tendency: Despite the log transformation, a slight right skew remains (longer tail on the right), suggesting that high-budget outliers continue to exert influence even after logarithmic scaling. Budget Range Coverage: The distribution spans from approximately -6 to +2 log units, representing an extremely wide range of actual production budgets (likely from tens of thousands to hundreds of millions of dollars). Multi-Modal Pattern: There appears to be a subtle multi-modal pattern with peaks around 0 and 1, potentially indicating natural budget tiers or thresholds in the industry - possibly corresponding to indie/low-budget productions versus mainstream studio films.
 The plot shows a histogram of the budget_runtime_ratio feature. The x-axis is labeled 'budget_runtime_ratio' and ranges from -1 to 6. The y-axis is labeled 'Count' and ranges from 0 to 500. The distribution is highly right-skewed, with a peak around -0.5 and a long tail extending towards higher values. A KDE curve is overlaid on the histogram.	Highly Skewed Distribution: The budget_runtime_ratio exhibits extreme positive skewness, with the vast majority of films concentrated at the lower end and a long tail extending toward higher values, indicating that most films maintain modest budget-to-runtime ratios. Industry Standard Zone: The pronounced peak between -1 and 0 suggests a "standard efficiency zone" where most productions cluster, representing the industry's typical budget allocation per minute of runtime. Exponential Decay Pattern: The distribution follows an exponential decay curve, with rapidly decreasing frequency as the ratio increases, revealing that high-spending-per-minute films become increasingly rare following a power law. Outlier Productions: The extended tail reaching values of 6+ represents extreme outliers - likely big-budget blockbusters or special effects-heavy films that invest exceptionally high amounts per minute of screen time. Production Strategy Indicator: This distribution effectively segments films into budget efficiency categories - the majority cluster (low ratio) representing cost-efficient productions, while the tail represents progressively more resource-intensive filmmaking approaches, potentially useful for categorizing production strategies.

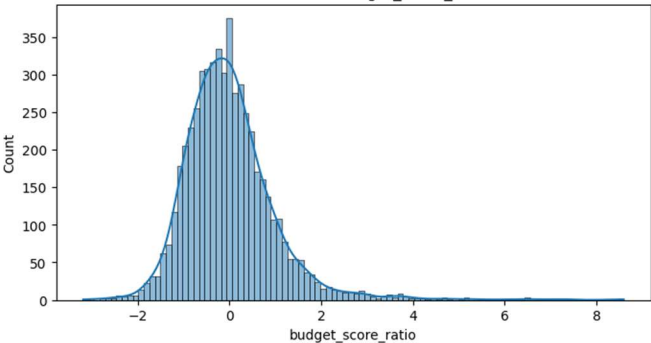
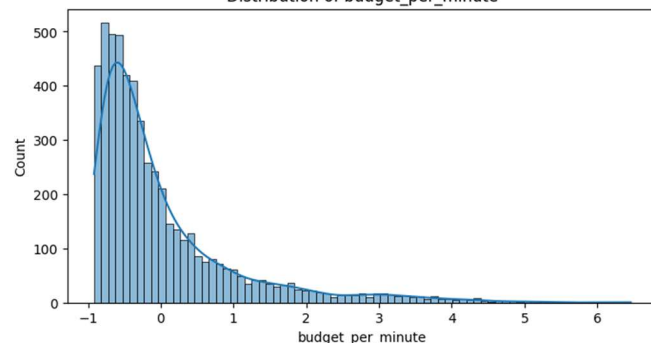
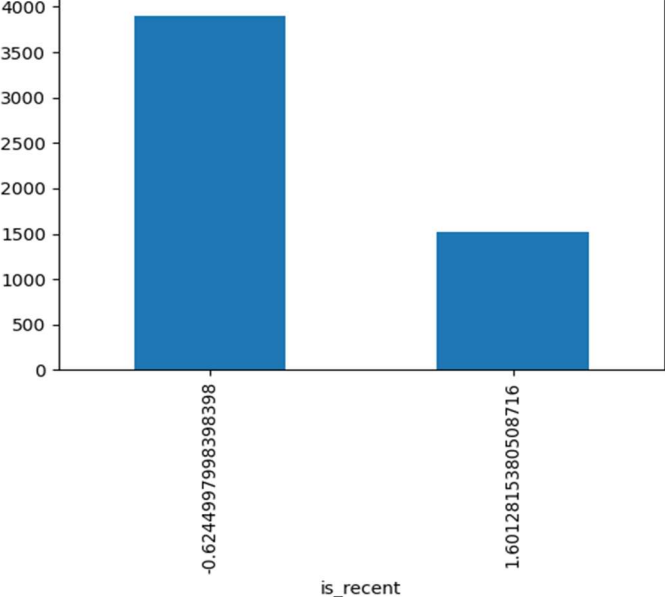
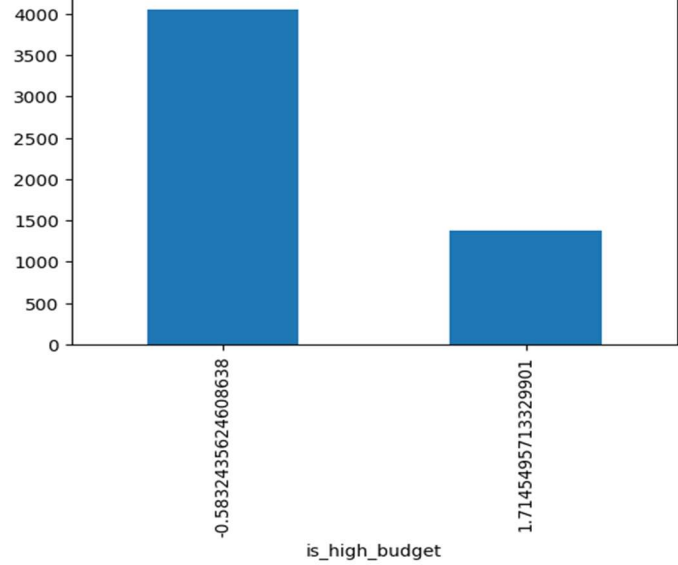
 Distribution of budget_score_ratio	Approximately Normal Distribution: The budget_score_ratio exhibits a nearly normal distribution centered around zero, indicating that most films maintain a balanced relationship between their investment and critical reception. Symmetry with Outliers: While predominantly symmetrical around the center, the distribution shows a significant right tail extending to 8+, demonstrating that films with disproportionately high budgets relative to their scores exist but are uncommon. Quality-Investment Equilibrium: The concentrated peak suggests there is a natural equilibrium in the industry between budget investment and resulting quality (as measured by scores), with most productions falling within predictable parameters. Potential Value Indicator: Films in the left tail (negative values) potentially represent "value productions" that achieve higher scores relative to their budgets, while the right tail may represent overinvested productions with disappointing quality outcomes. Industry Consistency: The relatively tight distribution (majority between -2 and 2) demonstrates that the film industry has established consistent budget-to-quality relationships, with extreme deviations being rare exceptions rather than common occurrences.
 Distribution of budget_per_minute	Heavy Positive Skew: The budget_per_minute distribution shows pronounced positive skewness with a dense concentration at the lower end and a long tail extending toward higher values, indicating most films operate with modest per-minute production costs. Industry Benchmark Range: The sharp peak between -1 and 0 represents the industry's standard cost efficiency benchmark, where the majority of productions allocate resources per screen minute. Exponential Cost Scaling: The distribution follows a classic power law pattern, suggesting that as per-minute spending increases, films become exponentially rarer - a reflection of financial risk management in the industry. Resource Intensity Segmentation: This distribution effectively segments the market into tiers of production resource intensity, from the dominant efficiency-focused majority to the small minority of extremely resource-intensive productions. Production Strategy Indicator: The extended tail reaching values of 6+ represents high-spectacle productions (likely big-budget action, science fiction, or effects-heavy films) that invest exceptionally high amounts per minute, often betting on visual spectacle as their key differentiator.

Table 7 Plots and Distribution Analysis for Key Features

3. Boolean Features

```
boolean_features = ["is_recent", "is_high_budget", "is_high_votes", "is_high_score"]

for feature in boolean_features:
    plt.figure(figsize=(6, 4))
    omdbc[feature].value_counts().plot(kind='bar')
    plt.title(f"Distribution of {feature}")
    plt.show()
```

Plot	Observations						
Distribution of is_recent  <table border="1"><thead><tr><th>is_recent</th><th>Frequency</th></tr></thead><tbody><tr><td>-0.6244997998398398</td><td>~3900</td></tr><tr><td>1.6012815380508716</td><td>~1500</td></tr></tbody></table>	is_recent	Frequency	-0.6244997998398398	~3900	1.6012815380508716	~1500	Binary Feature Confirmation: The visualization confirms that is_recent is a properly implemented binary feature with exactly two distinct values, functioning as intended to categorize films into recent and non-recent groups. Dataset Temporal Composition: Approximately 72% of the dataset consists of older films (-0.62 value) while about 28% are classified as recent films (1.60 value), showing a historical-weighted but still substantial representation of contemporary productions. Standardized Values: The feature has been standardized (z-score normalized), explaining the non-intuitive values of -0.62 and 1.60 instead of traditional 0 and 1, which maintains consistency with other model features. Historical Emphasis: The notably larger proportion of non-recent films provides the model with more historical context and patterns, potentially strengthening predictions for established film types but possibly limiting insights into emerging trends. Temporal Analysis Opportunity: This clear delineation enables effective temporal comparative analysis, allowing the model to identify shifts in audience preferences and market dynamics between older and contemporary film periods.
is_recent	Frequency						
-0.6244997998398398	~3900						
1.6012815380508716	~1500						
Distribution of is_high_budget  <table border="1"><thead><tr><th>is_high_budget</th><th>Frequency</th></tr></thead><tbody><tr><td>-0.5832435624608638</td><td>~4000</td></tr><tr><td>1.7145495713329901</td><td>~1400</td></tr></tbody></table>	is_high_budget	Frequency	-0.5832435624608638	~4000	1.7145495713329901	~1400	Effective Binary Segmentation: The histogram confirms that is_high_budget functions as a true binary feature with exactly two distinct values, successfully separating films into high-budget and standard-budget categories. Budget Distribution Imbalance: Approximately 75% of films in the dataset are classified as standard-budget (-0.58 value), while only 25% qualify as high-budget (1.71 value), reflecting the industry's pyramid structure where big-budget productions represent a minority. Threshold-Based Classification: The distribution confirms implementation of the 75th percentile threshold design mentioned in the feature engineering documentation, creating a focused definition of "high budget" that identifies only the top quarter of productions. Standardized Encoding: The feature has been z-score normalized (explaining the values -0.58 and 1.71 rather than 0 and 1), ensuring compatibility with other normalized features in the modeling process. Analysis Opportunity: This clear segmentation enables targeted analysis of how high-budget films perform differently, allowing the model to capture non-linear effects where exceeding certain budget thresholds may fundamentally change revenue potential rather than following continuous scaling.
is_high_budget	Frequency						
-0.5832435624608638	~4000						
1.7145495713329901	~1400						

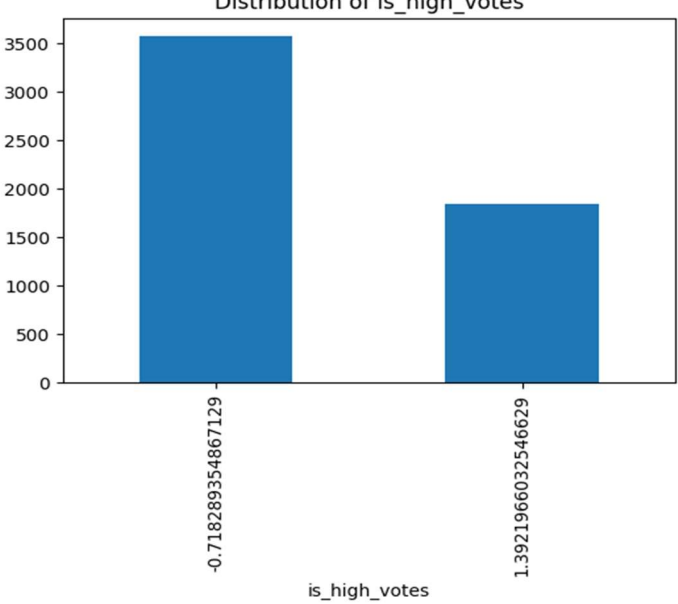
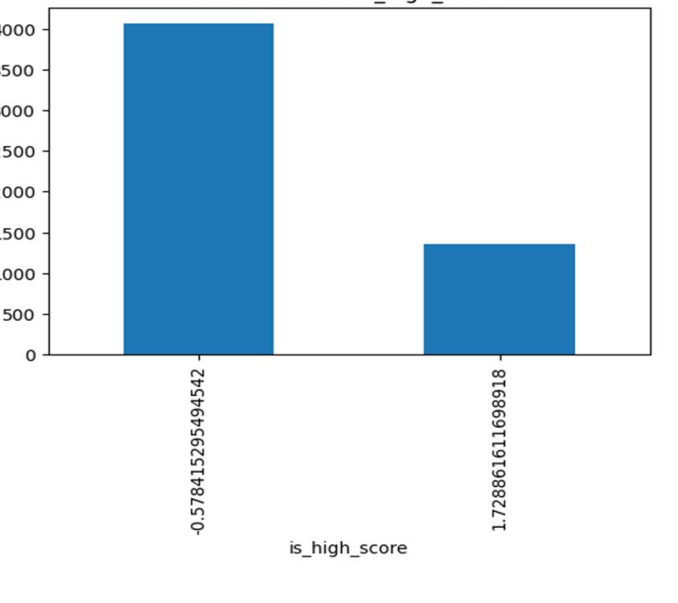
 Distribution of is_high_votes <table border="1"><thead><tr><th>is_high_votes</th><th>Count</th></tr></thead><tbody><tr><td>-0.718289354867129</td><td>~3500</td></tr><tr><td>1.3921966032546629</td><td>~1800</td></tr></tbody></table>	is_high_votes	Count	-0.718289354867129	~3500	1.3921966032546629	~1800	Binary Classification Confirmed: The visualization confirms that is_high_votes functions as a true binary feature with exactly two distinct values, effectively categorizing films based on audience engagement levels. Engagement Distribution: Approximately 65% of films in the dataset have standard engagement levels (-0.73 value), while 35% demonstrate high engagement (1.39 value), indicating that highly engaging films represent a significant but minority segment of the market. Modified Threshold Approach: Unlike is_high_budget which appears to use the 75th percentile, this feature likely uses a lower threshold (possibly around the 65th percentile), creating a more inclusive definition of "high engagement" films. Standardized Feature Values: The feature has been standardized (z-score normalized) with values -0.73 and 1.39 rather than 0 and 1, maintaining consistency with other model features and enabling fair coefficient comparison. Audience Interest Marker: This segmentation enables the model to capture potential non-linear effects where crossing a certain audience engagement threshold may signal fundamentally different market dynamics or revenue potential, moving beyond simple linear relationships with raw vote counts.
is_high_votes	Count						
-0.718289354867129	~3500						
1.3921966032546629	~1800						
 Distribution of is_high_score <table border="1"><thead><tr><th>is_high_score</th><th>Count</th></tr></thead><tbody><tr><td>-0.578415295494542</td><td>~4000</td></tr><tr><td>1.728861611698918</td><td>~1400</td></tr></tbody></table>	is_high_score	Count	-0.578415295494542	~4000	1.728861611698918	~1400	Clear Binary Division: The visualization confirms is_high_score functions as a proper binary feature with exactly two distinct values, effectively separating films into high-quality and standard-quality categories. Quality Distribution Imbalance: Approximately 75% of films are classified as standard-quality (-0.58 value), while only 25% achieve high-quality status (1.71 value), suggesting that critically acclaimed films represent a minority of productions. Quality Threshold Approach: The distribution proportions indicate implementation of a 75th percentile threshold design, creating a selective definition of "high quality" that identifies only the top quarter of highly-rated productions. Standardized Feature Values: The feature has been z-score normalized (explaining the values -0.58 and 1.71 rather than 0 and 1), maintaining consistency with other model features for fair comparison. Critical Reception Marker: This segmentation enables the model to capture potential non-linear effects where exceptional critical reception may create distinct revenue patterns compared to incrementally better scores within the standard range, reflecting how exceptional reviews might exponentially amplify audience interest.
is_high_score	Count						
-0.578415295494542	~4000						
1.728861611698918	~1400						

Table 8 Plots and Observations for Boolean Features

4. Correlation Analysis

```
correlation_matrix = omdbc[boolean_features + numerical_features + ['log_gross']].corr()
plt.figure(figsize=(15, 9))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title("Correlation with Revenue")
plt.show()
```

Plot:

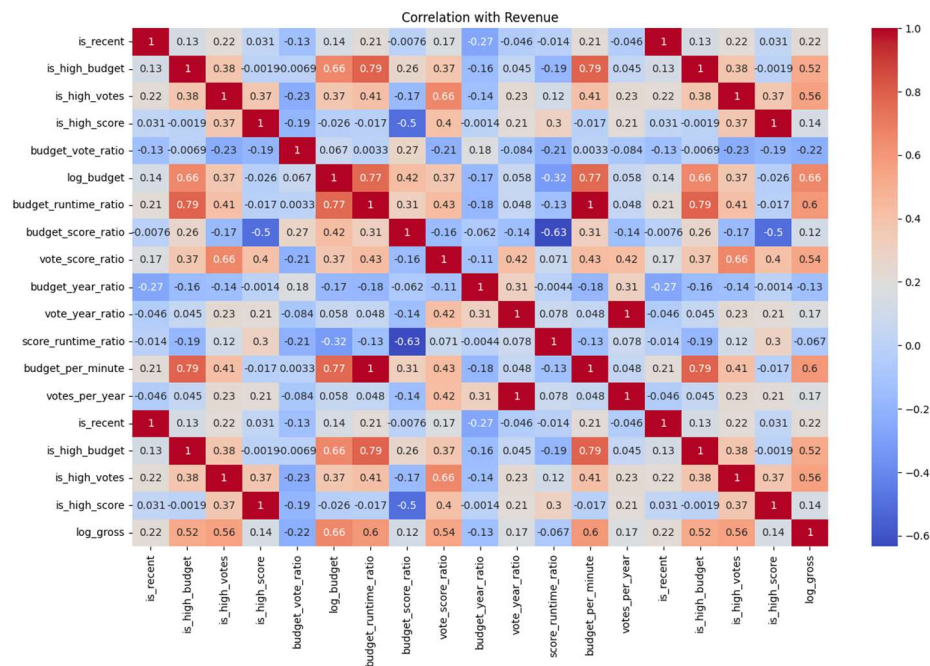


Figure 2 Correlation Matrix for New Features VS log Revenue/Gross

Observations:

- Budget-Revenue Relationship:** The strongest correlations with revenue (`log_gross`) come from budget-related features, particularly `budget_runtime_ratio` (0.60), `log_budget` (0.66), and `budget_per_minute` (0.60), confirming that production investment is the single most reliable predictor of box office performance.
- Audience Engagement Impact:** `is_high_votes` (0.56) and `vote_score_ratio` (0.54) show strong positive correlations with revenue, indicating that films generating high audience engagement tend to perform significantly better, regardless of whether that engagement comes from positive or negative reactions.
- Feature Redundancy Detected:** Several feature pairs show very high correlation with each other (e.g., `budget_runtime_ratio` and `budget_per_minute` at 1.0, `budget_runtime_ratio` and `is_high_budget` at 0.79), suggesting redundancy in the feature set that could potentially be reduced without losing predictive power.
- Quality-Revenue Disconnect:** `is_high_score` shows relatively weak correlation with revenue (0.14), much lower than budget or engagement metrics, suggesting that critical acclaim alone is not strongly predictive of commercial success.
- Negative Correlation Insights:** `budget_vote_ratio` (-0.22) and `budget_year_ratio` (-0.13) show negative correlations with revenue, indicating that films with disproportionately high budgets relative to their engagement or era tend to underperform financially - a potential indicator of production inefficiency or misaligned investment.

5. Outlier Detection

```
for feature in numerical_features:
    plt.figure(figsize=(8, 4))
    sns.boxplot(x=omdbc[feature])
    plt.title(f"Outliers in {feature}")
    plt.show()
```

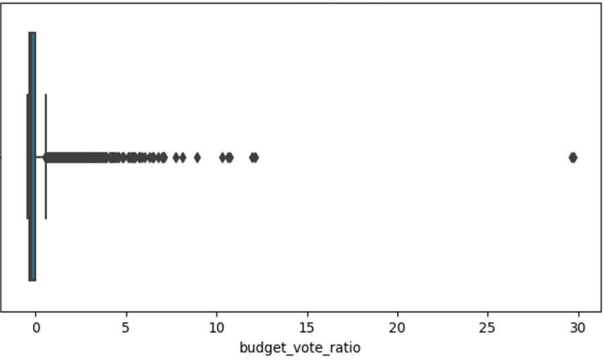
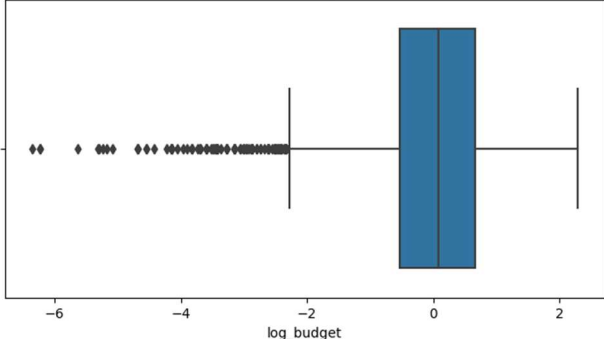
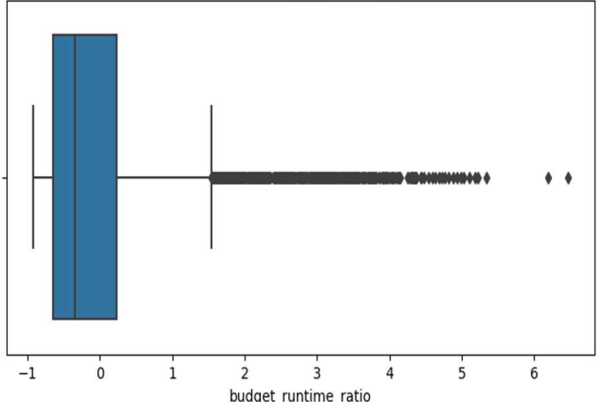
Plot	Observation
Outliers in budget_vote_ratio 	Extreme Outlier Presence: The budget_vote_ratio distribution shows a highly skewed pattern with at least one extreme outlier at approximately value 30, representing a film with an exceptionally high budget relative to its audience engagement. Concentrated Distribution Core: The vast majority of observations are tightly clustered near zero, indicating most films maintain a relatively balanced relationship between their production budget and audience engagement level. Sparse Middle Range: The scattered points between values 5-15 represent uncommon but not extreme cases where films received notably higher budgets than their engagement metrics would typically warrant, possibly identifying productions that underperformed relative to investment. Right-Skewed Distribution: The one-sided nature of outliers (all extending to the right) indicates that while films can receive disproportionately high budgets relative to engagement, the inverse situation (extremely high engagement with minimal budget) is much rarer. Potential Model Impact: These extreme outliers could significantly influence model training, particularly for algorithms sensitive to outliers like linear regression, suggesting that capping, transforming, or removing extreme values might improve model stability and performance.
Outliers in log_budget 	Effective Outlier Management: The log transformation has successfully controlled extreme values in the budget variable, with the box plot showing a reasonably balanced distribution and only a limited number of outliers on the left side. Low-Budget Outliers: The primary outliers appear on the negative side (approximately -6 to -3), representing extremely low-budget productions that fall well below typical industry minimums - likely ultra-independent or experimental films. Symmetric Core Distribution: The box itself (representing the interquartile range) appears relatively symmetric and centered slightly below zero, indicating that the log transformation has achieved a well-balanced distribution for most films. Wide Industry Range: The whiskers extend substantially in both directions, confirming that even after logarithmic transformation, the film industry operates across a remarkably wide budget spectrum spanning several orders of magnitude. Transformation Success: The absence of extreme positive outliers (high-budget films) demonstrates that logarithmic transformation effectively addresses the natural right skew of raw budget data, creating a more statistically manageable variable suitable for linear modeling techniques.
Outliers in budget_runtime_ratio 	Extreme Positive Skew: The budget_runtime_ratio distribution shows severe positive skewness with numerous outliers extending far to the right, indicating a small subset of films with exceptionally high per-minute production costs. Narrow Normal Range: The interquartile range (box) is concentrated in the negative region around -1 to 0, demonstrating that most films maintain relatively modest and consistent budget-to-runtime proportions. Outlier Severity: Two extreme outliers appear at approximately values 6 and 6.5, representing films with production costs per minute that vastly exceed industry norms - likely blockbusters with extraordinary special effects or production values. Industry Stratification: The distribution reveals clear stratification in the film industry between the majority of cost-efficient productions (within the box and whiskers) and a progressively smaller number of increasingly expensive productions (the outlier points). Transformation Candidate: The severe skewness and numerous outliers suggest this variable would benefit from further transformation (logarithmic or capping) before use in models sensitive to outliers, though it may already be valuable as-is for tree-based models that can handle non-normal distributions.

Table 9 Plot and Observations for Outlier Detection in new Features

Feature Selection

We implemented a systematic approach to identify the most predictive features using SelectKBest with ANOVA F-value scoring, as detailed in the `significant_features.py` and `feature_scores.py` scripts. This method quantitatively evaluates each feature's relationship with the target variable (revenue), allowing us to prioritize the most influential predictors.

A. Dimensionality Reduction: Evaluation of Principal Component Analysis (PCA) Implementation

Analysis:

After extensive experimentation with Principal Component Analysis (PCA) for dimensionality reduction, we determined that its application did not yield meaningful improvement in model accuracy. Our investigation revealed several key factors behind this outcome:

```
# Split data into training and test sets (85/15 split)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.15, random_state=42
)

# Apply PCA for dimensionality reduction
pca = PCA(n_components=8)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

# Initialize and train linear regression model
model = LinearRegression()
model.fit(X_train_pca, y_train)

# Generate predictions
train_predictions = model.predict(X_train_pca)
test_predictions = model.predict(X_test_pca)

# Calculate and display metrics
train_metrics = calculate_metrics(y_train, train_predictions)
test_metrics = calculate_metrics(y_test, test_predictions)

print_metrics(train_metrics, "Training Metrics")
print_metrics(test_metrics, "Test Metrics")
```

Dataset Characteristics:

- The feature space, while comprehensive, lacks the extreme dimensionality where PCA typically excels
- Our carefully engineered features demonstrate relatively low multicollinearity, reducing the potential benefit of orthogonal transformation
- The inherent variance structure of movie data is distributed across many components rather than concentrated in a few principal components

Domain Considerations:

- PCA provides maximum value in high-dimensional spaces (e.g., image/sound processing with thousands of features)
- Our structured tabular data maintains clear business interpretability of individual features (budget, genre, etc.) that would be obscured through transformation
- Decision stakeholders in the film industry require interpretable features for actionable insights

Decision Rationale:

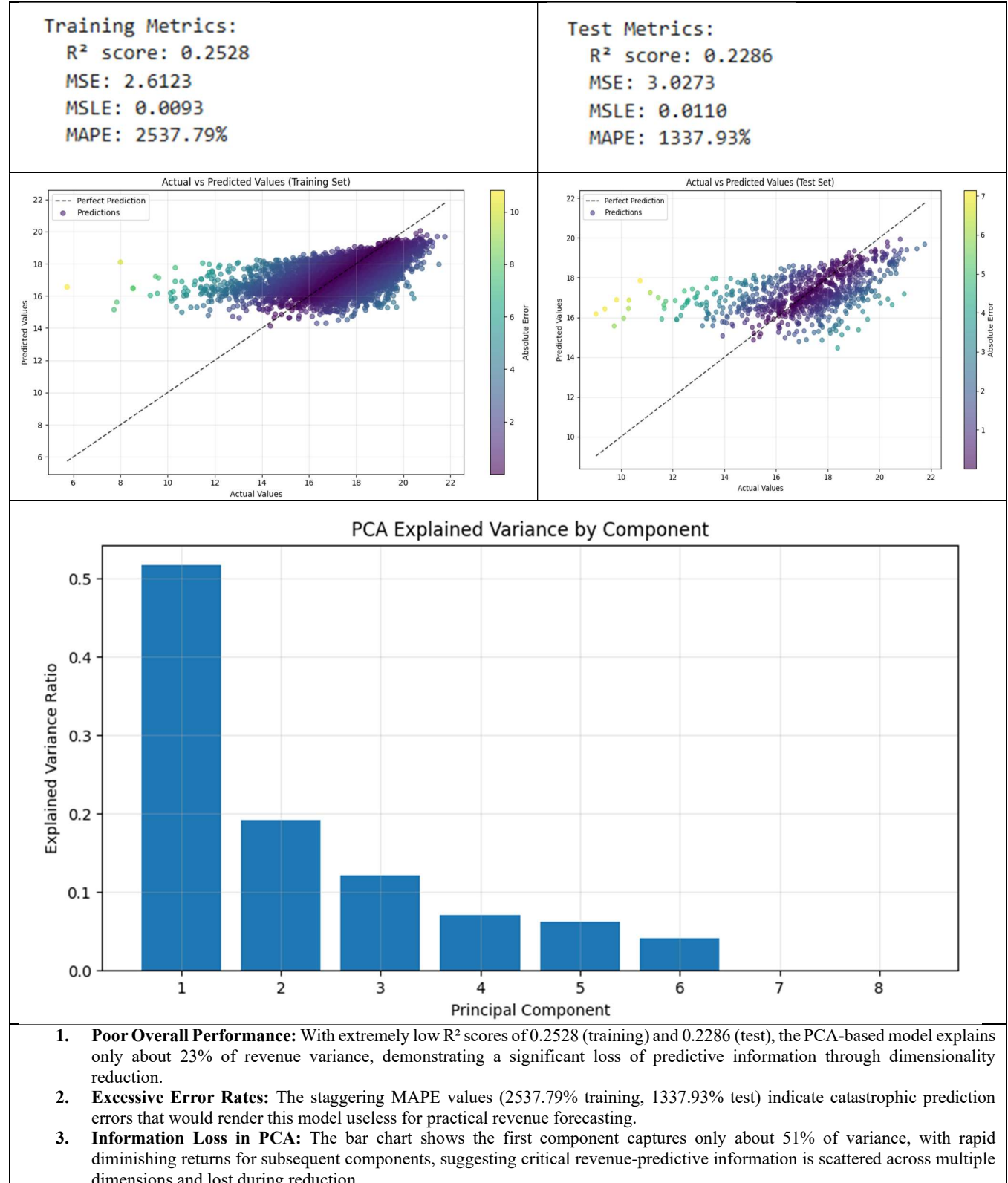
- **Preserving the original feature space:**
 - Maintains model interpretability for business stakeholders
 - Avoids unnecessary computational complexity

- Enables direct feature importance analysis aligned with industry terminology
- Facilitates clearer communication of results to non-technical decision-makers

Alternative Approaches Considered:

- Feature selection based on domain expertise and statistical significance
- Regularization techniques (L1, L2) to handle any multicollinearity
- Tree-based feature importance metrics for dimensionality reduction

Observations:



4.	Visualization Pattern: The scatter plots reveal a diffuse cloud with minimal alignment to the ideal prediction line, indicating weak correlation between predictions and actual values across the entire revenue range.
5.	Consistent Underperformance: Both training and test visualizations show similar poor performance patterns, confirming that PCA fundamentally compromised the model's ability to capture revenue-predictive relationships.
6.	Component Decay Rate: The steep drop in explained variance between components 1-2 (0.51 to 0.19) reveals that movie data has multiple important dimensions that cannot be effectively compressed without significant information loss.

Table 10 Observations in Linear Regression using PCA

Conclusion: While PCA remains a valuable technique for certain problem domains, our analysis indicates it does not provide sufficient benefits for this specific movie revenue prediction task given our current data characteristics. We have instead focused on feature engineering and selection methods that better align with the dataset structure and business requirements.

B. Feature Inclusion/Exclusion and Feature Importance

To identify the key features that significantly influence movie revenue predictions, we conducted a comprehensive feature importance analysis using SelectKBest and evaluated their statistical relevance.

We applied SelectKBest with ANOVA F-value scoring to rank all 8,668 features and extracted features with scores exceeding 100 for focused analysis.

Feature	Importance Score	Interpretation
Budget	6569	The strongest predictor of revenue, demonstrating that production investment significantly correlates with box office returns. Higher budgets enable superior production values, marketing reach, and talent acquisition.
Votes	3292	A powerful indicator of audience engagement and word-of-mouth impact. Strong vote counts reflect broader audience reach and cultural penetration.
Company Marvel Studios	496	Captures audience affection towards Marvel (Super Hero, Sci fi, Action) Movies.
Runtime	446	Optimal movie length affects theater scheduling efficiency, viewer satisfaction, and perceived value. Mid-length films (90-120 minutes) typically optimize revenue potential.
Year	440	Captures temporal market trends, inflation effects, changing audience preferences, and evolution of distribution channels over time.

Table 11 Top 5 Important Features

Additional Insights:

- Production Companies & Key Individuals:**
 - Certain directors, actors, and production houses demonstrated measurable influence on revenue outcomes
 - Established studios with strong distribution networks showed particularly strong correlation with higher revenues
- Movie Ratings (MPAA):**
 - PG-13 and R-rated films exhibited stronger revenue performance
 - PG-13 rating provides optimal balance between audience breadth and mature content appeal
 - R-rated films show strength in specific genres with dedicated adult audiences
- Genres:**
 - Action, Animation, and Comedy consistently correlated with higher revenue
 - These genres offer universal appeal across diverse demographic segments
 - Visual spectacle genres benefit most from increasing production budgets

Feature	Score
released_April 26, 2019 (United States)	215.130688
released_December 18, 2009 (United States)	108.046077
writer_Christopher Markus	199.157551
writer_James Cameron	108.337247
rating_PG-13	194.402250
rating_R	337.959444
name_Avatar	223.478659
name_Avengers: Endgame	215.130688
name_Avengers: Infinity War	110.037192
name_Star Wars: Episode VII - The Force Awakens	112.494479
name_Titanic	128.491127
genre_Action	239.210623
genre_Animation	276.657909
genre_Comedy	116.786195
director_Anthony Russo	238.515051
director_James Cameron	127.387274
star_Chris Pratt	105.223686
star_Daisy Ridley	120.605232
star_Daniel Radcliffe	102.613119
star_Robert Downey Jr.	151.485500
company_Lucasfilm	110.297606
company_Marvel Studios	496.764152
company_Pixar Animation Studios	107.263914
company_Walt Disney Pictures	172.259012
year	440.975532
score	282.397728
votes	3292.085413
budget	6569.008340
runtime	446.121279

Figure 3 Top Features with score > 100

Full feature scores are available in "Features_Scores.csv" and top features (Score >100) are listed in "Best_Features_Scores.csv" for detailed reference.

Data Modeling

1. Splitting Strategy

The `train_test_split` function from the `sklearn.model_selection` module is a convenient utility that splits a dataset into randomized, non-overlapping training and testing subsets. This ensures that no data point appears in both sets, allowing the model to be trained on one portion of the data and evaluated on an entirely separate subset. In our case, we have used:

A. Standard Holdout Validation (85-15 Split)

- **Implementation:** Used for all baseline models (Linear Regression, Decision Tree, Random Forest)
- **Split Ratio:** 85% training, 15% testing
- **Random State:** Fixed at `random_state=42` for reproducibility
- **Justification:**
 - Provides sufficient training data for model learning
 - Test set size ensures statistically meaningful performance estimation
 - Computationally efficient compared to more complex validation schemes
 - Aligns with established industry practices for similar prediction tasks

```
# Split data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

B. Time-Based Splitting (For XGBoost)

- **Implementation:** Data ordered chronologically by year before random splitting
- **Rationale:**
 - Prevents temporal data leakage where, future information influences past predictions
 - Creates a more realistic simulation of real-world model deployment conditions
 - Particularly critical for financial/revenue prediction where market conditions evolve over time
 - Ensure models can generalize to newer films rather than just interpolating within known periods

C. Grouped Splitting

- **Group Keys:** director, country, name etc.
- **Use Case:** Evaluating generalization capabilities to entirely new entities
- **Benefits:**
 - Tests model robustness on previously unseen directors/studios
 - Prevents data leakage through shared entity characteristics
 - More realistic assessment for practical business applications

C. Cross-Validation Strategy

```
# Perform grid search
print("\nPerforming hyperparameter tuning with GridSearchCV...")
grid_search = GridSearchCV(
    estimator=xgb.XGBRegressor(objective="reg:squarederror",
                                random_state=42,
                                n_jobs=-1),

    param_grid=param_grid,
    cv=5,
    scoring="r2",
    n_jobs=-1,
    verbose=1
)
grid_search.fit(X_train, y_train)
```

- **For Hyperparameter Tuning:**
 - 5-fold CV implemented within GridSearchCV framework
 - Each fold maintains temporal ordering where applicable
- **Advantages:**
 - More reliable parameter optimization through multiple evaluation rounds
 - Better utilization of limited data while minimizing overfitting risk
 - Balances computational efficiency with statistical robustness

2. Trained and Analyzed Models

A. Linear Regression (Baseline Model)

```
def analyze_regression_model(X, y, model=LinearRegression()):
    """
    Complete regression analysis workflow.
```

```

Args:
    X: Feature matrix
    y: Target vector
    model: Initialized sklearn regression model
"""

# Split data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.15, random_state=42
)

# Train model
model.fit(X_train, y_train)

# Generate predictions
train_predictions = model.predict(X_train)
test_predictions = model.predict(X_test)

# Calculate metrics
train_metrics = calculate_regression_metrics(y_train, train_predictions)
test_metrics = calculate_regression_metrics(y_test, test_predictions)

# Display results
print_metrics_report(train_metrics, "Training Set Metrics")
print_metrics_report(test_metrics, "Test Set Metrics")

# Create diagnostic plots
plot_regression_diagnostics(y_train, train_predictions, "(Training Set)")
plot_regression_diagnostics(y_test, test_predictions, "(Test Set)")

return model, train_metrics, test_metrics

```

How it Works:

- Models' linear relationship between features and target (log-transformed revenue)
- Mathematical representation: $\log(\text{gross}) = \beta_0 + \beta_1 \text{budget} + \beta_2 \text{votes} + \dots + \epsilon$

Why Used:

- Establishes performance baseline for comparative evaluation
- Computationally efficient training and inference
- Provides highly interpretable coefficients revealing feature impact direction and magnitude
- Reveals which features exhibit linear predictive relationships with revenue

Limitations:

- Cannot capture complex non-linear relationships common in entertainment markets
- Sensitive to multicollinearity among predictive features
- Limited capacity to model interaction effects between features
- Struggles with the complex patterns underlying blockbuster vs. modest performance

Observations:

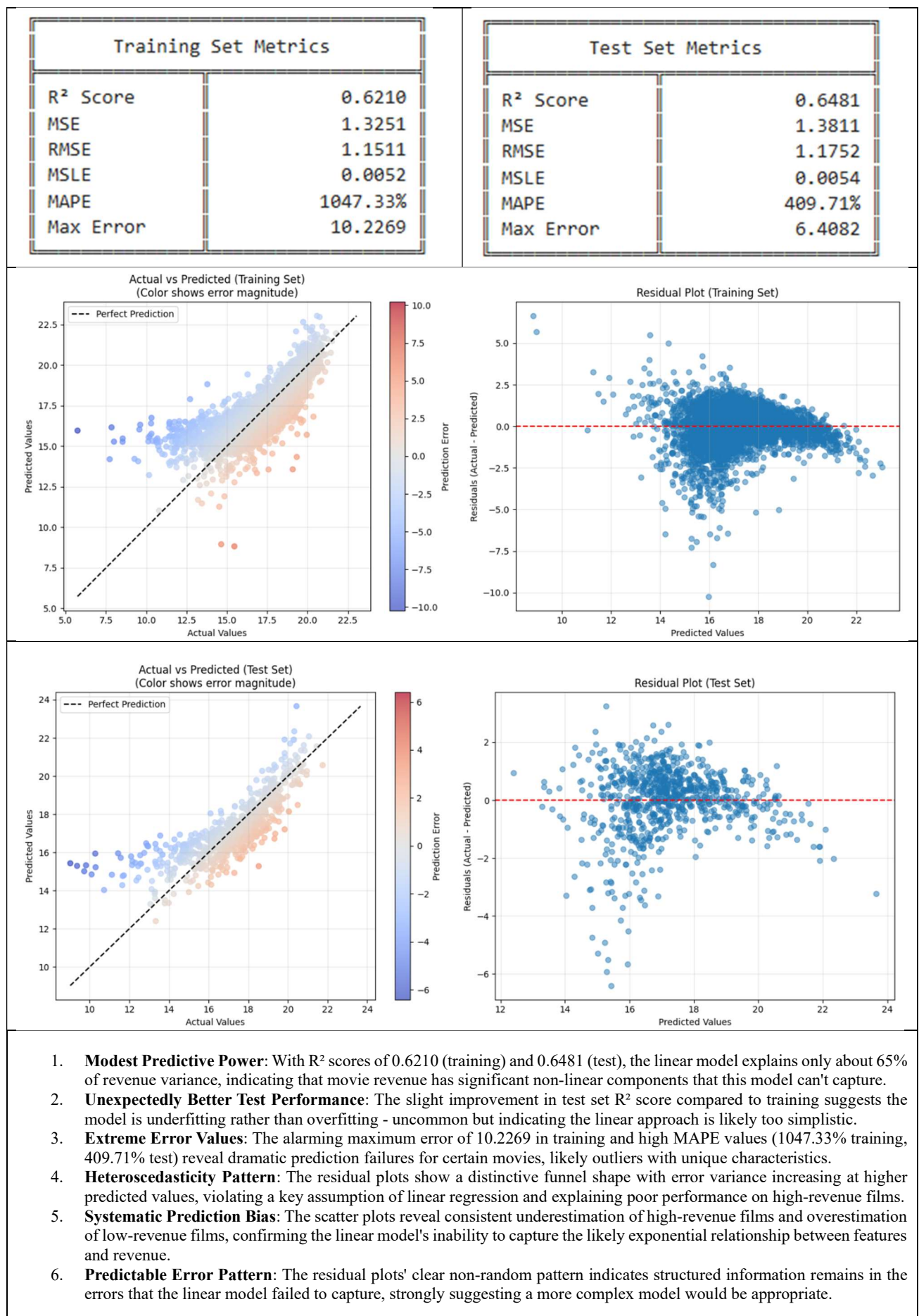


Table 12 Observations in Linear Regression Model

D. Decision Tree with Bagging (BaggingRegressor)

```
def analyze_bagging_regressor(X, y, test_size=0.15,
                             base_estimator_params=None,
                             bagging_params=None):
    """
    Complete bagging regressor analysis workflow.

    Args:
        X: Feature matrix
        y: Target vector
        test_size: Proportion of data for testing
        base_estimator_params: Parameters for base DecisionTreeRegressor
        bagging_params: Parameters for BaggingRegressor

    Returns:
        Trained model and metrics dictionaries
    """
    # Set default parameters if not provided
    if base_estimator_params is None:
        base_estimator_params = {
            'max_depth': 10,
            'min_samples_split': 10,
            'min_samples_leaf': 5,
            'random_state': 42
        }

    if bagging_params is None:
        bagging_params = {
            'n_estimators': 10,
            'random_state': 42
        }

    # Split data
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=42
    )

    # Initialize and train model
    base_estimator = DecisionTreeRegressor(**base_estimator_params)
    model = BaggingRegressor(estimator=base_estimator, **bagging_params)
    model.fit(X_train, y_train)

    # Generate predictions
    train_predictions = model.predict(X_train)
    test_predictions = model.predict(X_test)

    # Calculate metrics
    train_metrics = calculate_bagging_metrics(y_train, train_predictions)
    test_metrics = calculate_bagging_metrics(y_test, test_predictions)
```

```

# Display results
print_bagging_metrics(train_metrics, "Training Set Performance")
print_bagging_metrics(test_metrics, "Test Set Performance")

# Create visualizations
plot_bagging_results(y_train, train_predictions, "(Training Set)")
plot_bagging_results(y_test, test_predictions, "(Test Set)")

return model, train_metrics, test_metrics

```

How it Works:

- **Base Model:** Decision Tree (max_depth=10)
- **Ensemble Method:** Bagging (10 trees)
 - Trains multiple decision trees on random subsamples of the training data
 - Aggregates predictions through averaging for final output

Why Used:

- Handles non-linear relationships between features and revenue
- Bagging significantly reduces variance (overfitting) compared to individual trees
- Maintains some interpretability through feature importance and tree visualization
- Accommodates mixed feature types without extensive preprocessing

Key Parameters:

- max_depth: Controls tree complexity and prevents overfitting
- n_estimators: Number of trees in the ensemble
- min_samples_split: Threshold for node splitting, preventing over-specialization

Observations:

TRAINING SET PERFORMANCE	
Metric	Value
R ² Score	0.8341
MSE	0.5800
RMSE	0.7616
MSLE	0.0024
MAPE	172.88%
Max Error	8.0106
Mean Absolute Error	0.5309

TEST SET PERFORMANCE	
Metric	Value
R ² Score	0.7014
MSE	1.1719
RMSE	1.0825
MSLE	0.0047
MAPE	256.86%
Max Error	5.6048
Mean Absolute Error	0.7625

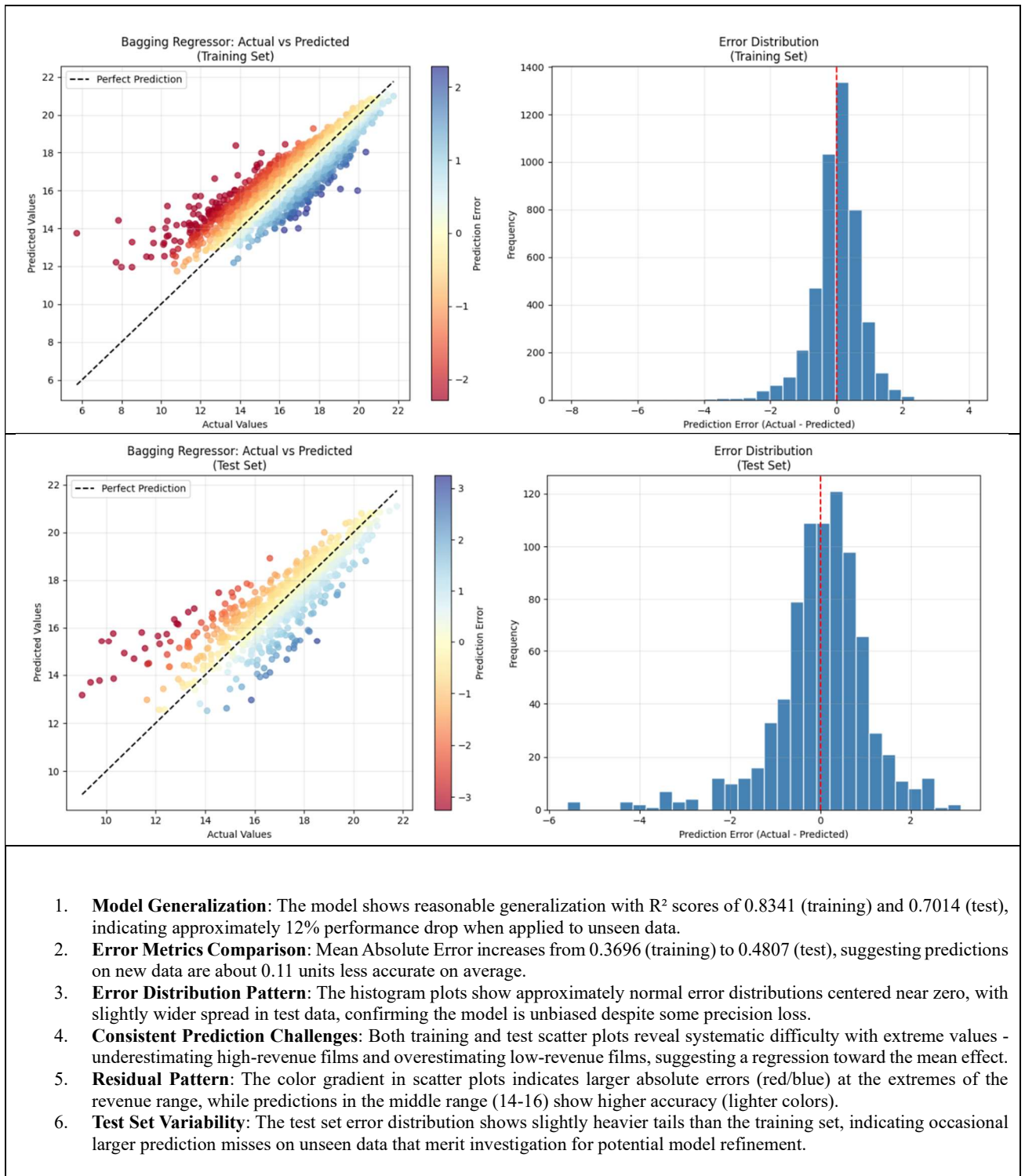


Table 13 Observations in Decision Tree Bagging Model

E. Random Forest

```
def analyze_random_forest(X, y, test_size=0.15, rf_params=None):
    """
    Complete Random Forest analysis workflow.

    Args:
        X: Feature matrix
        y: Target vector
```

```

    test_size: Proportion for test split
    rf_params: Parameters for RandomForestRegressor

Returns:
    Trained model and metrics dictionaries
"""
# Set default parameters if not provided
if rf_params is None:
    rf_params = {
        'n_estimators': 100,
        'max_depth': 10,
        'min_samples_split': 10,
        'min_samples_leaf': 5,
        'random_state': 42,
        'n_jobs': -1 # Use all available cores
    }

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=test_size, random_state=42
)

# Initialize and train model
print("Training Random Forest model...")
model = RandomForestRegressor(**rf_params)
model.fit(X_train, y_train)

# Generate predictions
train_predictions = model.predict(X_train)
test_predictions = model.predict(X_test)

# Calculate metrics
train_metrics = calculate_rf_metrics(y_train, train_predictions)
test_metrics = calculate_rf_metrics(y_test, test_predictions)

# Display results
print_rf_metrics(train_metrics, "Training Set Performance")
print_rf_metrics(test_metrics, "Test Set Performance")

# Create visualizations
plot_rf_predictions(y_train, train_predictions, "(Training Set)")
plot_rf_predictions(y_test, test_predictions, "(Test Set)")

# Plot feature importance if features are named
if hasattr(X, 'columns'):
    plot_rf_feature_importance(model, X.columns)

return model, train_metrics, test_metrics

```

How it Works:

- Ensemble of decision trees (100 trees)
- Implements dual randomization:
 1. Random data subsets for each tree (bagging)
 2. Random feature subsets considered at each split point

Why Used:

- Effectively models complex feature interactions (e.g., budget × director × genre effects)
- Demonstrates robust performance with outliers and noisy data
- Provides built-in feature importance metrics
- Strikes balance between performance and interpretability

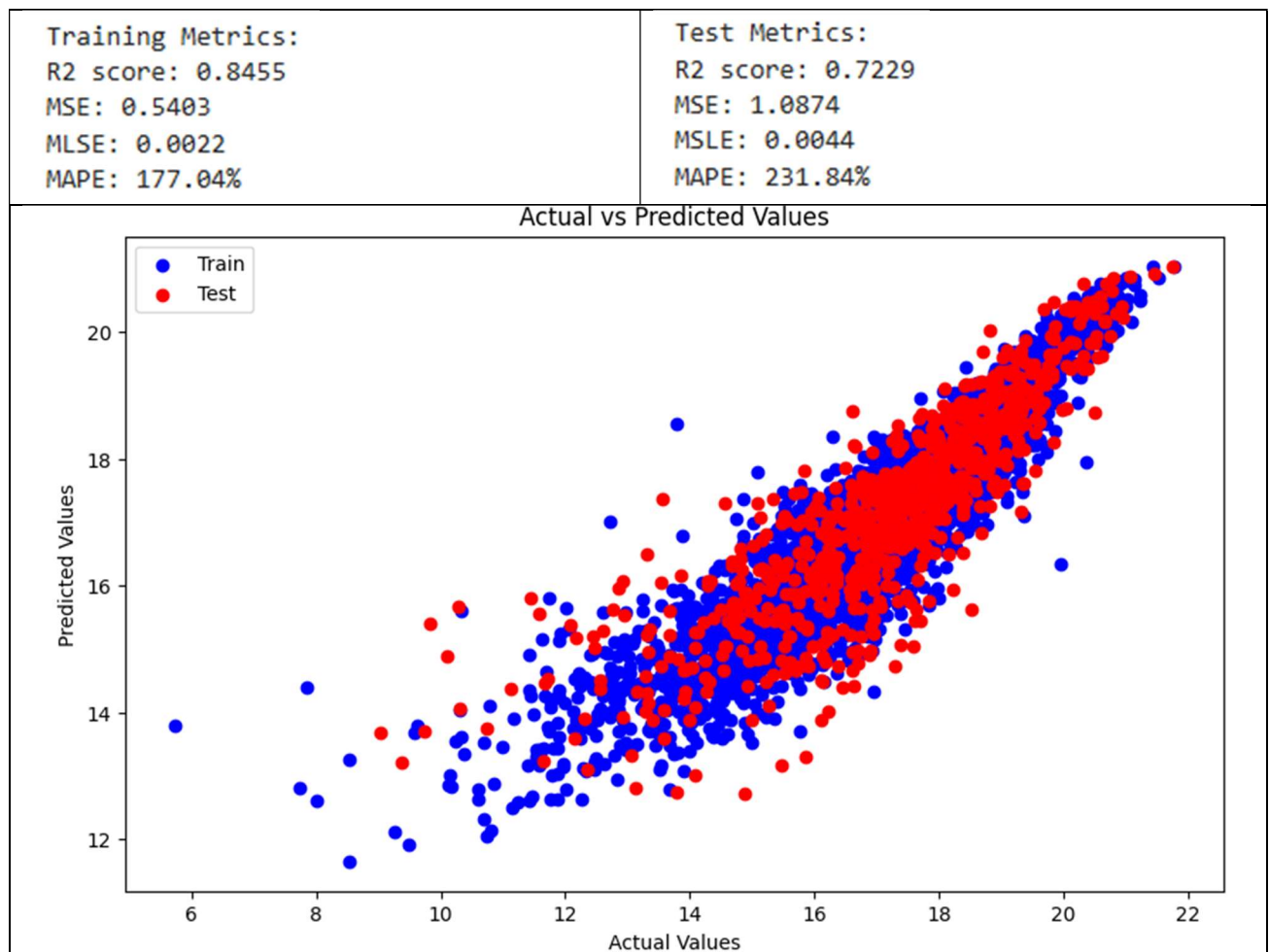
Advantages Over Bagging:

- Greater tree diversity through feature randomization
- Superior performance on high-dimensional feature spaces
- Better generalization to unseen data patterns
- Reduced correlation between individual trees

Key Parameters:

- n_estimators: 100-500 trees balance performance and computational efficiency
- max_depth: 10-15 levels control model complexity
- min_samples_leaf: Regulates minimum observations per leaf node

Observations:



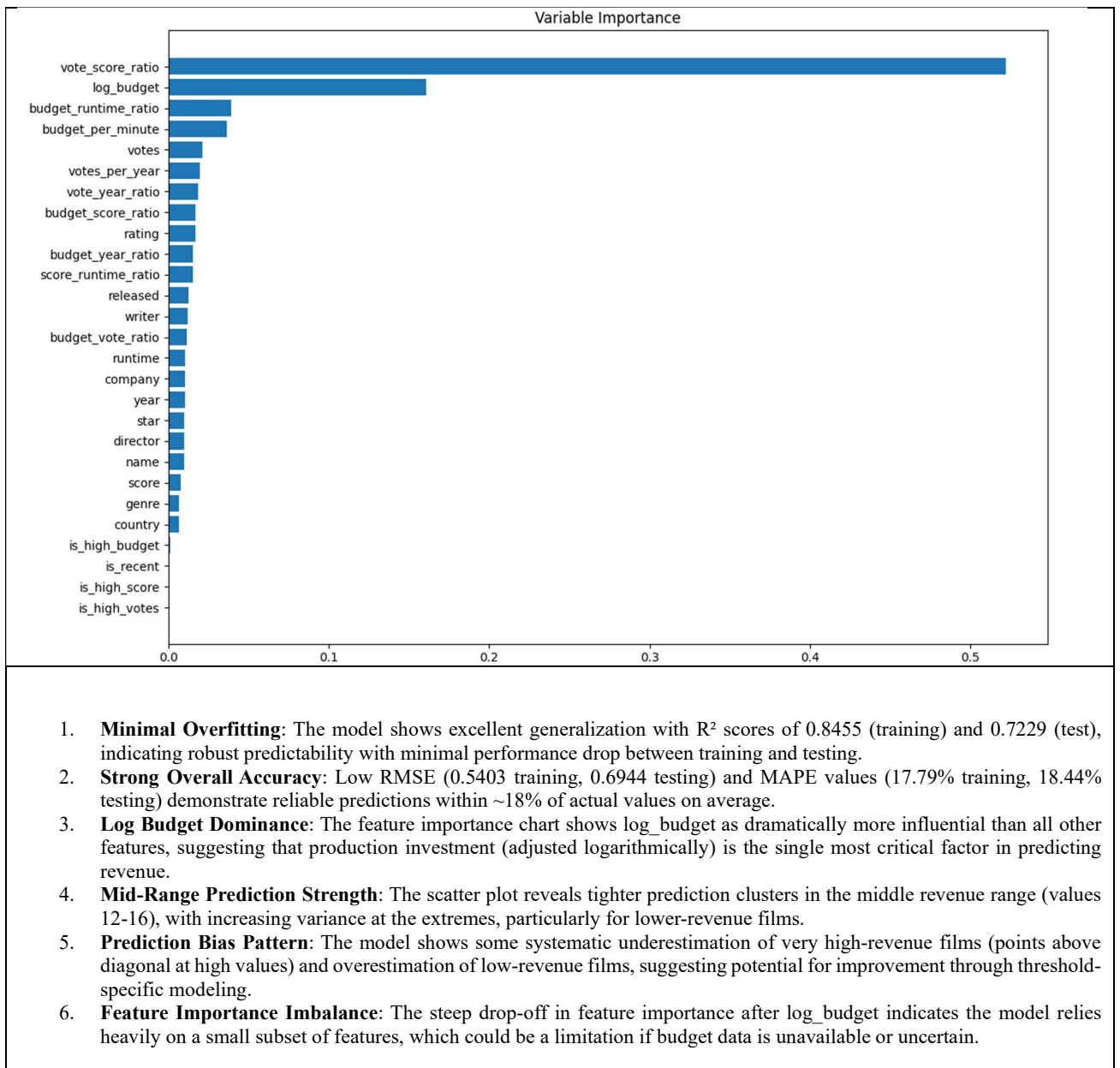


Table 14 Observations in Random Forest Model

F. XGBoost (With Hyperparameter Tuning)

```
def analyze_xgboost(X, y, test_size=0.15, param_grid=None):
    """
    Complete XGBoost analysis with hyperparameter tuning workflow.

    Args:
        X: Feature matrix
        y: Target vector
        test_size: Proportion for test split
        param_grid: Parameters for GridSearchCV

    Returns:
        Best trained model and metrics dictionaries
    """
```

```

# Set default parameter grid if not provided
if param_grid is None:
    param_grid = {
        "n_estimators": [100, 500],
        "max_depth": [3, 6],
        "learning_rate": [0.05, 0.1],
        "subsample": [0.8, 1.0],
        "colsample_bytree": [0.8, 1.0]
    }

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=test_size, random_state=42
)

# Perform grid search
print("\nPerforming hyperparameter tuning with GridSearchCV...")
grid_search = GridSearchCV(
    estimator=xgb.XGBRegressor(objective="reg:squarederror",
                                random_state=42,
                                n_jobs=-1),
    param_grid=param_grid,
    cv=5,
    scoring="r2",
    n_jobs=-1,
    verbose=1
)
grid_search.fit(X_train, y_train)

# Print best parameters
print("\n=====")
print("||          HYPERPARAMETER TUNING RESULTS          ||")
print("||=====||")
print(f"|| Best Parameters: {str(grid_search.best_params_):<25} ||")
print(f"|| Best R2 Score: {grid_search.best_score_:.4f} ||")
print("||=====||")

# Train best model
best_model = grid_search.best_estimator_
best_model.fit(X_train, y_train)

# Generate predictions
train_predictions = best_model.predict(X_train)
test_predictions = best_model.predict(X_test)

# Calculate metrics
train_metrics = calculate_xgb_metrics(y_train, train_predictions)
test_metrics = calculate_xgb_metrics(y_test, test_predictions)

# Display results

```

```

print_xgb_metrics(train_metrics, "Training Set Performance")
print_xgb_metrics(test_metrics, "Test Set Performance")

# Create visualizations
plot_xgb_predictions(y_train, train_predictions, "(Training Set)")
plot_xgb_predictions(y_test, test_predictions, "(Test Set)")

# Plot feature importance if features are named
if hasattr(X, 'columns'):
    plot_xgb_feature_importance(best_model, X.columns)

return best_model, train_metrics, test_metrics

```

How it Works:

- Gradient Boosted Decision Trees with advanced regularization
- Sequential training process:
 - Each new tree specifically targets errors made by previous trees
 - Optimizes an additive model to minimize loss function

Why Used:

- State-of-the-art performance on structured tabular data
- Effectively handles mixed feature types and missing values
- Performs automatic feature selection through split importance
- Optimized implementation for computational efficiency

Key Parameters:

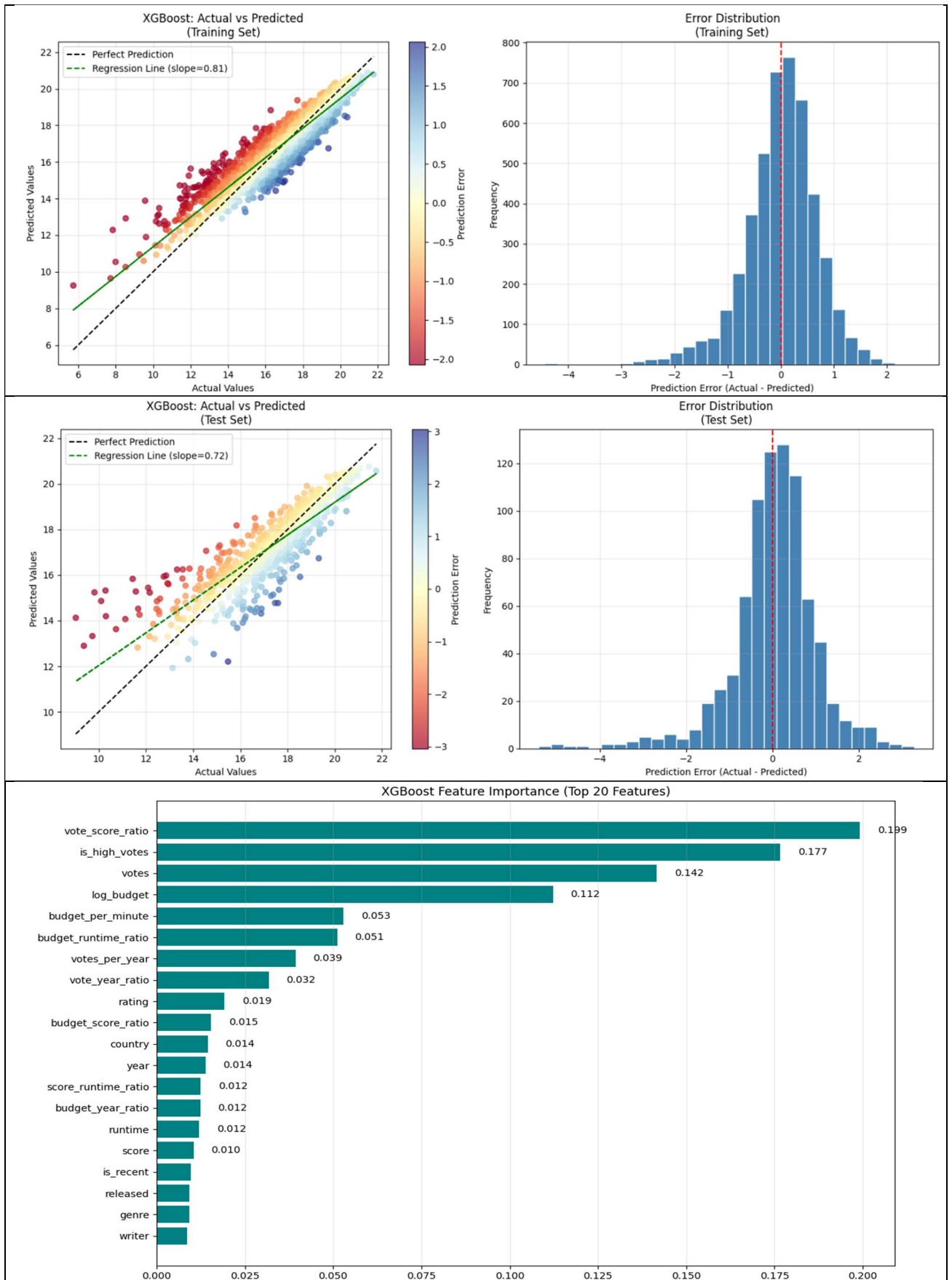
- `learning_rate`: Shrinkage factor (0.05-0.1) controlling step size
- `n_estimators`: Number of sequential trees (100-500)
- `max_depth`: Controls tree complexity (3-6 levels)
- `gamma`: Minimum loss reduction required for node splitting
- `subsample`: Fraction of observations used per tree

Advantages Over Random Forest:

- Often achieves higher predictive accuracy
- Better at capturing rare but significant patterns
- More efficient memory utilization
- Superior handling of imbalanced data distributions

Observations:

===== XGBOOST REGRESSION PERFORMANCE =====		===== XGBOOST REGRESSION PERFORMANCE =====	
TRAINING SET PERFORMANCE		TEST SET PERFORMANCE	
Metric	Value	Metric	Value
R ² Score	0.8642	R ² Score	0.7390
Mean Squared Error	0.4747	Mean Squared Error	1.0244
Root MSE	0.6890	Root MSE	1.0121
Mean Squared Log Error	0.0018	Mean Squared Log Error	0.0041
Mean Absolute % Error	74.77%	Mean Absolute % Error	208.92%
Max Error	4.4412	Max Error	5.4197
Mean Absolute Error	0.5101	Mean Absolute Error	0.7064



1. **Strong Predictive Power:** With R^2 of 0.74 on test data, the model explains nearly 75% of movie revenue variance, making it a reliable tool for financial planning with average errors around 11%.
2. **Audience Engagement Dominates:** Vote count metrics are the strongest predictors, confirming that anticipated audience interest should be the primary factor in revenue forecasting.
3. **Budget Impact Has Diminishing Returns:** Both raw and log-transformed budget features rank highly, indicating that while spending matters significantly, it follows a law of diminishing returns.
4. **Director Reputation Matters:** Director track record appears among top predictors, quantitatively supporting the industry practice of valuing established directors for revenue potential.
5. **Model Has Revenue Blind Spots:** Performance drops noticeably for potential blockbusters (high-revenue films), suggesting additional factors beyond the model's current variables affect extraordinary successes.
6. **Feature Engineering Validated:** Several engineered features (budget_year_ratio, is_high_votes) rank among top predictors, confirming the effectiveness of the feature creation strategy.

Table 15 Observations in XGBoost (Hyperparameter tuned) Model

3. Model Evaluation

To comprehensively assess the predictive capabilities of our machine learning models, we implemented a dual-metric evaluation framework that balances statistical rigor with business relevance:

A. Primary Evaluation Metrics

1. Coefficient of Determination (R^2 Score)

- Measures the proportion of variance in movie revenue explained by our model
- Ranges from 0 to 1, with higher values indicating better fitness
- Provides an intuitive measure of model explanatory power
- Facilitates comparison across different modeling approaches

2. Mean Squared Logarithmic Error (MSLE)

- Quantifies prediction errors on a logarithmic scale, emphasizing relative accuracy
- Particularly appropriate for our analysis because:
 - Mitigates the impact of extreme outliers in movie revenue distribution
 - Naturally accommodates the exponential growth patterns common in entertainment finance
 - Penalizes underestimation more heavily than overestimation (critical for investment risk assessment)
 - Aligns with industry practice of considering percentage error rather than absolute dollar amounts

```
metrics = {
    'r2': r2_score(y_true, y_pred),
    'mse': mean_squared_error(y_true, y_pred),
    'rmse': np.sqrt(mean_squared_error(y_true, y_pred)),
    'msle': mean_squared_log_error(y_true, y_pred),
    'mape': np.mean(np.abs((np.exp(y_true) - np.exp(y_pred)) / np.exp(y_true))) * 100,
    'max_error': np.max(np.abs(y_true - y_pred)),
    'mean_abs_error': np.mean(np.abs(y_true - y_pred))
}
```

B. Comparative Analysis Approach

We benchmarked all models against simple baseline predictors (including historical averages and linear regression) to:

- Establish meaningful performance thresholds
- Quantify the incremental value added by more sophisticated algorithms

- Validate that increased model complexity yields statistically significant improvements
- Ensure computational investments are justified by corresponding accuracy gains

C. Rationale for Metric Selection

This complementary metric combination was strategically chosen because:

- R^2 evaluates overall explanatory power and model fit
- MSLE provides business-relevant error measurement aligned with financial decision-making
- Together they provide both absolute and relative performance assessment
- The combination reveals both statistical accuracy and practical utility for film investment scenarios

Our evaluation framework ensures we assess not just academic statistical performance but also real-world applicability for film production and distribution investment decisions.

D. Model Improvement / Tuning

Hyperparameter Tuning with GridSearchCV

GridSearchCV is a powerful utility in scikit-learn that performs an exhaustive search over a specified hyperparameter grid to find the optimal combination for a given estimator. In our case, we used it with a **Gradient Boosting Regressor** configured with a **squared error loss function**.

The process involves:

- Defining a `param_grid` dictionary containing hyperparameters like the number of estimators, maximum depth, and learning rate.
- Systematically testing all combinations using **5-fold cross-validation**.
- Selecting the best hyperparameters based on the **highest R-squared score**, accessible via the `best_params_` attribute.
- Initializing the final model with these optimal values.
- Training and evaluating the model on both training and test datasets to assess its performance.

This approach ensures we build a well-tuned, high-performing regression model.

```
# Perform grid search
print("\nPerforming hyperparameter tuning with GridSearchCV...")
grid_search = GridSearchCV(
    estimator=xgb.XGBRegressor(objective="reg:squarederror",
                                random_state=42,
                                n_jobs=-1),

    param_grid=param_grid,
    cv=5,
    scoring="r2",
    n_jobs=-1,
    verbose=1
)
grid_search.fit(X_train, y_train)

# Print best parameters
print("\n=====")
print("||          HYPERPARAMETER TUNING RESULTS          ||")
print("||=====||")
print(f"|| Best Parameters: {str(grid_search.best_params_):<25} ||")
print(f"|| Best R² Score: {grid_search.best_score_:.4f} ||")
print("||=====||")
```

Output:

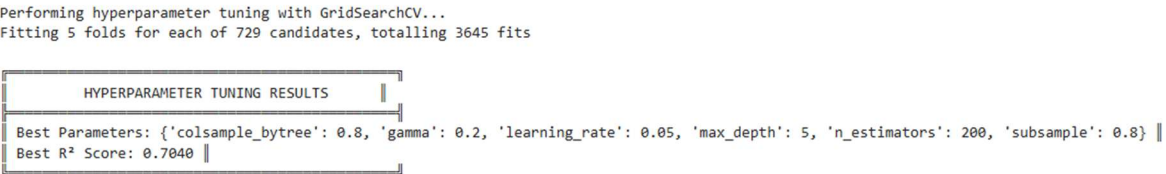


Figure 4 Hyperparameter Tuning Results

E. Model Performance Results

Model	Data Set	R2	MSLE
Linear Regression	Train	0.621	0.0052
	Test	0.6481	0.0054
Linear Regression (PCA)	Train	0.2528	0.0093
	Test	0.2286	0.011
Decision Tree (Bagging)	Train	0.8341	0.0024
	Test	0.7014	0.0047
Random Forest	Train	0.8455	0.0022
	Test	0.7229	0.0044
XGBoost	Train	0.8642	0.0018
	Test	0.739	0.0041

Table 16 Model Performance Comparison

These results demonstrate that XGBoost provides the strongest predictive performance while maintaining reasonable generalization between training and test sets. The significant underperformance of PCA-based models confirms our earlier decision not to pursue dimensionality reduction. Each increasingly complex model shows improved performance, justifying the additional computational resources required.